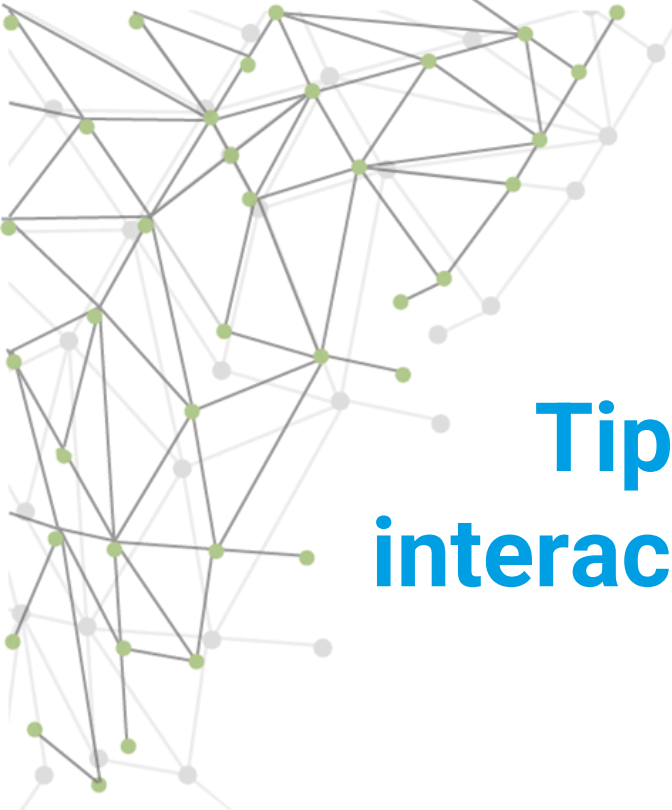




Tipos de métodos e interacciones entre objetos

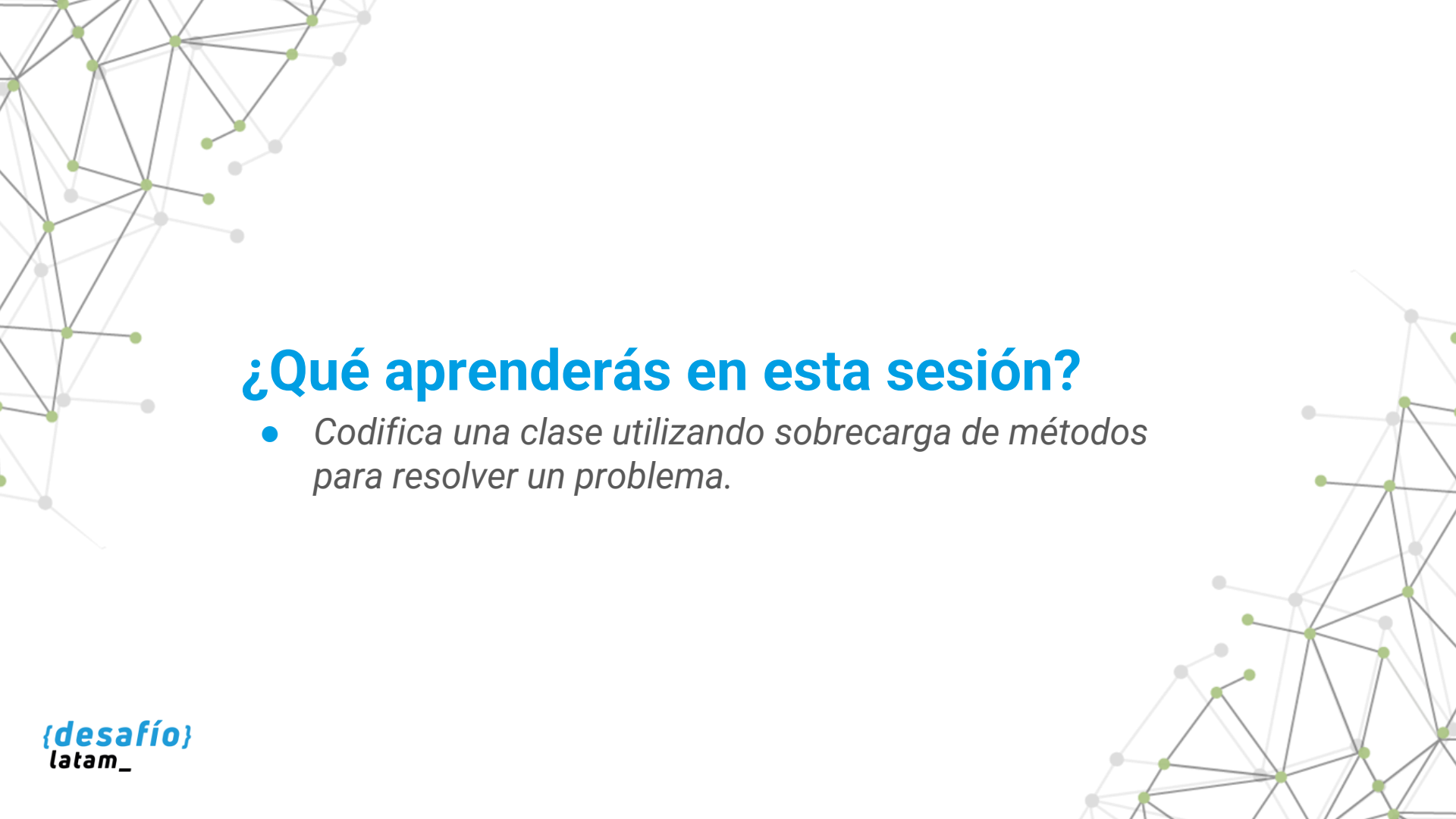
Sobrecarga de métodos

Codificar un programa con clases, atributos y métodos utilizando colaboración y composición para resolver un problema de baja complejidad acorde al lenguaje Python.

- Unidad 1:
Introducción a la programación orientada a objetos con Python
- Unidad 2:
Tipos de métodos e interacciones entre objetos
- Unidad 3:
Herencia y polimorfismo
- Unidad 4:
Manejo de errores y archivos



Te encuentras aquí



¿Qué aprenderás en esta sesión?

- *Codifica una clase utilizando sobrecarga de métodos para resolver un problema.*

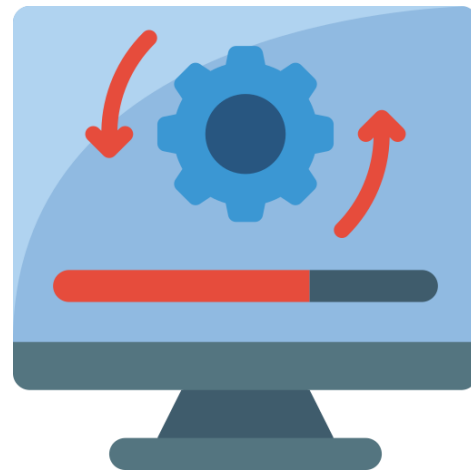
¿Qué entendemos por
sobrecarga de métodos?



**/* Sobrecarga y sobrescritura
de métodos */**

Sobrecarga

- Permite definir varios métodos con el mismo nombre pero con diferentes conjuntos de parámetros.
- Se basa en la firma del método, que incluye el nombre y los tipos de datos de los parámetros.
- Los métodos sobrecargados se seleccionan en tiempo de compilación según los argumentos proporcionados en la llamada al método.



Métodos especiales en Python

Categoría de métodos especiales	Ejemplos
Creación, eliminación y representación de objetos	<code>__init__</code> , <code>__del__</code> , <code>__str__</code>
Acceso a atributos	<code>__getattr__</code> , <code>__setattr__</code>
Atributos del descriptor	<code>__get__</code> , <code>__set__</code> , <code>__delete__</code>
Secuenciar y mapear objetos	<code>__len__</code> , <code>__getitem__</code> , <code>__contains__</code>
Iteración	<code>__iter__</code>
Operaciones matemáticas	<code>__add__</code> , <code>__pow__</code> , <code>__int__</code>
Comparaciones	<code>__lt__</code> , <code>__eq__</code> , <code>__ne__</code>
Llamado de objetos	<code>__call__</code>

Ejemplo de sobrecarga de método

```
class Dato:
    def __init__(self, dato1):
        self.dato1 = dato1

dato1 = Dato(10)
dato2 = Dato(30)

print(dato1 + dato2)
```

- En este código estamos intentando realizar la suma de dos instancias de la clase Dato.
- Esto arroja el siguiente error, dado que no podemos realizar operación de suma con el operador + en instancias de una clase.

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-13-53555700b3e6> in <cell line: 14>()
      12 dato2 = Dato(30)
      13
----> 14 print(dato1 + dato2)

TypeError: unsupported operand type(s) for +: 'Dato' and 'Dato'
```

Ejemplo de sobrecarga de método

Método especial `__add__` (suma)

- Añadimos el método especial `__add__` para definir la lógica de suma de datos.
- Además, añadimos el método especial `__str__` para formatear la salida de información al ejecutarse el código.



```
class Dato:
    def __init__(self, dato1):
        self.dato1 = dato1

    def __str__(self):
        return f"{self.dato1}"

    def __add__(self, suma_datos):
        return Dato(self.dato1 + suma_datos.dato1)

dato1 = Dato(10)
dato2 = Dato(30)

print(dato1 + dato2)
```

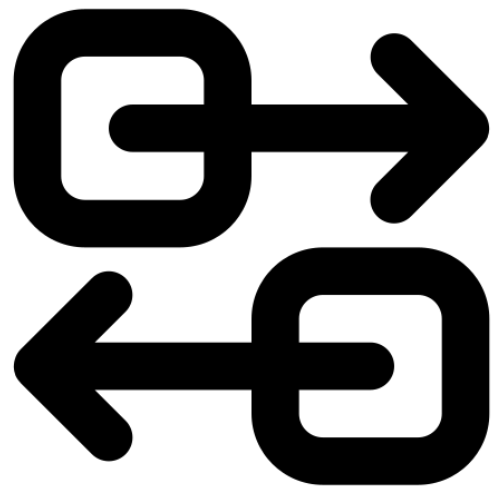
Ejercicio:

Aplicando los métodos especiales (`__sub__`), utiliza el código anterior y realiza la resta de las instancias de `Dato`.



Sobrescritura

- Ocurre cuando una subclase redefine un método heredado de una clase padre.
- El método sobrescrito debe tener el mismo nombre, tipo de retorno y lista de parámetros que el método de la clase padre.
- La sobrescritura se produce en tiempo de ejecución cuando se llama al método desde una instancia de la subclase.



Ejemplo de sobrescritura

- La sobrescritura se ejecuta en métodos de Python bajo el concepto de herencia.
- En el código tenemos una clase principal y una subclase que hereda de la principal.
- En este caso, la subclase hereda los métodos definidos.
- En la subclase se sobrescribe la salida de ambos métodos.

```
#Clase principal
class Monoplaza:
    color = ""

    def aceleracion(self):
        print("El monoplaza acelera")

    def frenado(self):
        print("El monoplaza frena")

#Subclase que -->hereda<-- de la principal
class Redbull(Monoplaza):
    def aceleracion(self):
        print("El monoplaza Redbull acelera hasta los 450km/h")

    def frenado(self):
        print("El monoplaza Redbull tiene buena capacidad de frenado")

redbull = Redbull()
redbull.aceleracion()
redbull.frenado()
```

¿Por qué se ejecuta el
método de la subclase y no la
de Monoplaza o la clase
principal?



- Esto sucede dado que Python va a ubicar dentro el método de la misma clase que se está instanciando.
- En este caso, la instancia se está realizando sobre la subclase.

```
#Clase principal
class Monoplaza:
    color = ""

    def aceleracion(self):
        print("El monoplaza acelera")

    def frenado(self):
        print("El monoplaza frena")

#Subclase que --hereda-- de la principal
class Redbull(Monoplaza):
    def aceleracion(self):
        print("El monoplaza Redbull acelera hasta los 450km/h")

    def frenado(self):
        print("El monoplaza Redbull tiene buena capacidad de frenado")

redbull = Redbull()
redbull.aceleracion()
redbull.frenado()
```

Ejercicio:

Genera nuevas subclases en el código de la clase Monoplaza y genera nuevas subclases. Un monoplaza es un vehículo tipo de competición Fórmula 1.



Ejercicio guiado

"Stock de medicamentos"



Stock de medicamentos

Desde la cadena farmacéutica para la cual has venido desarrollando un programa para manejar los medicamentos en venta, te solicitan esta vez crear un prototipo que permita manejar el stock de medicamentos a medida que son ingresados. En este contexto, el cliente especifica que el programa debe considerar lo siguiente:

- Para ingresar un medicamento se debe solicitar nombre y stock, y añadir a un listado de medicamentos ingresados.
- Dos medicamentos se consideran iguales si tienen el mismo nombre (insensible a mayúsculas).
- Solo se debe solicitar (y asignar) el precio (considerando lo mismo del programa anterior en cuanto a IVA, descuento y precio final) en caso de que se ingrese un medicamento que no existe.
- Si se ingresa un medicamento que ya existe, se debe agregar al stock del medicamento existente el del medicamento del nuevo ingreso.
- Luego de cada ingreso (de un medicamento nuevo o existente), se debe informar nombre del medicamento ingresado, precio bruto, descuento (si posee), precio final, stock, y cantidad de medicamentos distintos que tiene ingresados la farmacia.
- El programa debe permitir un nuevo ingreso de medicamento hasta que el usuario indique lo contrario.



Stock de medicamentos

Solución

Paso 1

A partir de la clase Medicamento del archivo **medicamento.py** del ejercicio guiado “Ingreso de medicamento”, sobrecarga el método **__eq__**, de forma que dos instancias de Medicamento se consideran iguales si sus nombres son iguales (independiente de mayúsculas y minúsculas).

```
class Medicamento():
    IVA = 0.18
    def __init__(self, nombre: str, stock: int = 0):
        self.nombre = nombre
        self.stock = stock
        self.precio_bruto = 0
        self.precio_final = 0.0
        self.descuento = 0.0
    @staticmethod
    def validar_mayor_a_cero(numero: int):
        return numero > 0
    @property
    def precio(self):
        return self.precio_final
    @precio.setter
    def precio(self, precio_bruto: int):
        if self.validar_mayor_a_cero(precio_bruto):
            self.precio_bruto = precio_bruto
            self.precio_final = precio_bruto + precio_bruto * self.IVA

            if self.precio_final >= 10000 and self.precio_final < 20000:
                self.descuento = 0.1

            elif self.precio_final >= 20000:
                self.descuento = 0.2

            if self.descuento:
                self.precio_final *= 1 - self.descuento

# Este es el método que se debe agregar
def __eq__(self, other):
    return self.nombre.lower() == other.nombre.lower()
```

Stock de medicamentos

Solución

Paso 2

A continuación del código anterior, sobrecargar el método `__iadd__` (permite sobrecargar operación de asignación `+=`), de forma que, al sumar dos instancias iguales de `Medicamento`, se añada al stock de la instancia actual el stock de la segunda instancia. El método debe retornar la instancia actual.

```
def __iadd__(self, other):  
    if self == other:  
        self.stock += other.stock  
    return self
```



Stock de medicamentos

Solución

Paso 3

En el archivo **programa.py**, luego de importar la clase Medicamento, consultar mediante input si el usuario desea ingresar un medicamento y almacenar la opción ingresada en una variable. También iniciar una lista vacía donde se irán almacenando los medicamentos ingresados.

```
# archivo programa.py
from medicamento import Medicamento
opcion_ingreso = int(input("¿Desea agregar un medicamento?"
"\n1. Sí\n2. No\n"))
ingresados = []
```



Stock de medicamentos

Solución

Paso 4

A continuación del código anterior, iniciar un ciclo while que tenga como condición de ingreso que la opción ingresada en el paso anterior sea 1. Dentro del ciclo, consultar nombre y stock, y crear una instancia de Medicamento con estos datos.

```
while opcion_ingreso == 1:
    nombre = input("\nIngrese nombre del medicamento:\n")
    stock = int(input("\nIngrese stock del medicamento:\n"))
    m = Medicamento(nombre, stock)
```

Stock de medicamentos

Solución

Paso 5

A continuación del código anterior, consultar si el nuevo medicamento se encuentra ya ingresado (Nota: el operador `in` desencadena un llamado a `__eq__`). De ser así, realizar la suma de stocks y reemplazar el medicamento ingresado por el existente.

```
if m in ingresados:
    indice = ingresados.index(m)
    ingresados[indice] += m
```



Stock de medicamentos

Solución

Paso 6

En caso de que no se cumpla la condición anterior, solicitar el precio, asignar y agregar el medicamento a la lista de ingresados.

```
else:  
    ingresados.append(m)  
    precio_bruto = int(input("\nIngrese precio bruto del medicamento:\n"))  
    m.precio = precio_bruto
```



Stock de medicamentos

Solución

Paso 7

Luego de finalizar el ingreso, muestre en pantalla los datos del medicamento ingresado, y la cantidad de medicamentos distintos ingresados.

```
print(f"\n***** DATOS MEDICAMENTO {m.nombre} *****")
print(f"PRECIO BRUTO: ${m.precio_bruto}")
if m.descuento:
    print(f"DESCUENTO: {m.descuento*100}%")
print(f"PRECIO FINAL: ${m.precio_final}")
print(f"STOCK: {m.stock}")

print(f"\nLa farmacia cuenta con {len(ingresados)} medicamento(s)\n")
```



Stock de medicamentos

Solución

Paso 8

Por último, al final del ciclo, consultar nuevamente si se desea ingresar o no un nuevo medicamento.

```
opcion_ingreso = int(input("¿Desea agregar un medicamento?"  
"\n1. Sí\n2. No\n"))
```



¿A qué se refiere la
sobrecarga en Python?



¿Qué permite la sobrecarga
de operadores?





Próxima sesión...

- *Desafío guiado.*

{desafío}
latam_

*Academia de
talentos digitales*

